Name : Gaurav Hinduja
Div : D15C
Batch : C
Roll No : 50

MLDL Experiment 02

Aim: Implement Multi Regression, Lasso, and Ridge Regression on real-world datasets.

1. Dataset Source:

- **Dataset Name:** Insurance Datasets
- **Source:** https://www.kaggle.com/datasets/mirichoi0218/insurance
- **Repository Owner:** Mirichi Choi

2. Dataset Description:

The Medical Insurance dataset is used to predict the **insurance charges** of a person based on demographic and health-related features.

- **Number of rows:** 1338
- **Number of columns:** 7
- **Target Variable:** `charges`

Input Features

1. **age** – age of the individual
2. **sex** – gender of the individual

3. **bmi** – body mass index
4. **children** – number of children/dependents covered by insurance
5. **smoker** – smoking status
6. **region** – residential region
7. **charges** – medical insurance cost (target)

This dataset is suitable for regression because the output variable `charges` is continuous.

3. Mathematical Formulation of the Algorithms:

A. Multi-Linear Regression (OLS):

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_n x_n$$

Goal: Minimize Mean Squared Error (MSE).

B. Ridge Regression (L2 Regularization):

Adds penalty term:

$$J(\beta) = \sum (y_i - \hat{y}_i)^2 + \lambda \sum \beta_j^2$$

- Shrinks coefficients
- Reduces overfitting
- Does NOT eliminate features

C. Lasso Regression (L1 Regularization):

Adds penalty term:

$$J(\beta) = \sum (y_i - \hat{y}_i)^2 + \lambda \sum |\beta_j|$$

- Can shrink coefficients to zero
- Performs feature selection

4. Algorithm Limitations:

- **Multi-Linear Regression:** Sensitive to multicollinearity and outliers.
- **Ridge Regression:** Reduces overfitting but keeps all features. ●

Lasso Regression: May remove important correlated variables.

5. Methodology / Workflow (Updated for Student Dataset):

1. Load the insurance dataset
2. Apply one-hot encoding to categorical columns using `pd.get_dummies()`
3. Separate input features and target variable (`charges`)
4. Split the dataset into training and testing sets
5. Standardize the feature values using `StandardScaler()`
6. Train Linear Regression, Lasso, and Ridge models
7. Evaluate the models using MAE, MSE, and R^2

8. Tune alpha values for Lasso and Ridge using `GridSearchCV`
9. Compare base models and tuned models
10. Plot coefficient comparison and predicted vs actual values

6.Performance Analysis:

Phase 1: Initial Model Comparison

In the first phase, the default versions of Linear Regression, Lasso, and Ridge were trained.

- **Linear Regression** gives a baseline model
- **Lasso** applies L1 regularization
- **Ridge** applies L2 regularization

Observation

The insurance dataset contains both numerical and categorical attributes. Features such as **smoker**, **BMI**, and **age** strongly affect the insurance charges. Regularization improves model stability and helps control overfitting.

Phase 2: Final Test Set Evaluation (After Tuning)

After tuning alpha values using GridSearchCV, Lasso and Ridge models were re-evaluated on the test set.

Overall Observation

The tuned models provide better generalization. In this dataset, **smoking status** is generally the most important feature affecting medical insurance charges.

7. Hyperparameter Tuning:

We applied **GridSearchCV with 5-Fold Cross-Validation** to find optimal regularization strength.

A. Lasso Regression Tuning

- Parameter Tuned: alpha
- Tested Values: [0.0001, 0.001, 0.01, 0.1, 1.0]

Best Results:

- Best Alpha: **1.0**
- Best Cross-Validation R^2: **0.7836**

Impact:

A higher alpha value indicates stronger regularization. This means some coefficients are reduced significantly, but overall performance remains similar to the base model, suggesting limited overfitting in the dataset.

B. Ridge Regression Tuning

- Parameter Tuned: alpha
- Tested Values: [0.1, 1.0, 10.0, 100.0]

GridSearch (Normal)

- Tested Values: [0.1, 1.0, 10.0, 100.0]
- Best Alpha: 10
- Test R^2: 0.7830

Logspace Tuning

- Best Alpha: ≈ 7.54
- Test R^2: 0.7831

Impact:

A moderate alpha (around 7–10) stabilizes coefficients and slightly improves model generalization. However, the performance difference is very small, indicating the dataset already has stable linear relationships.

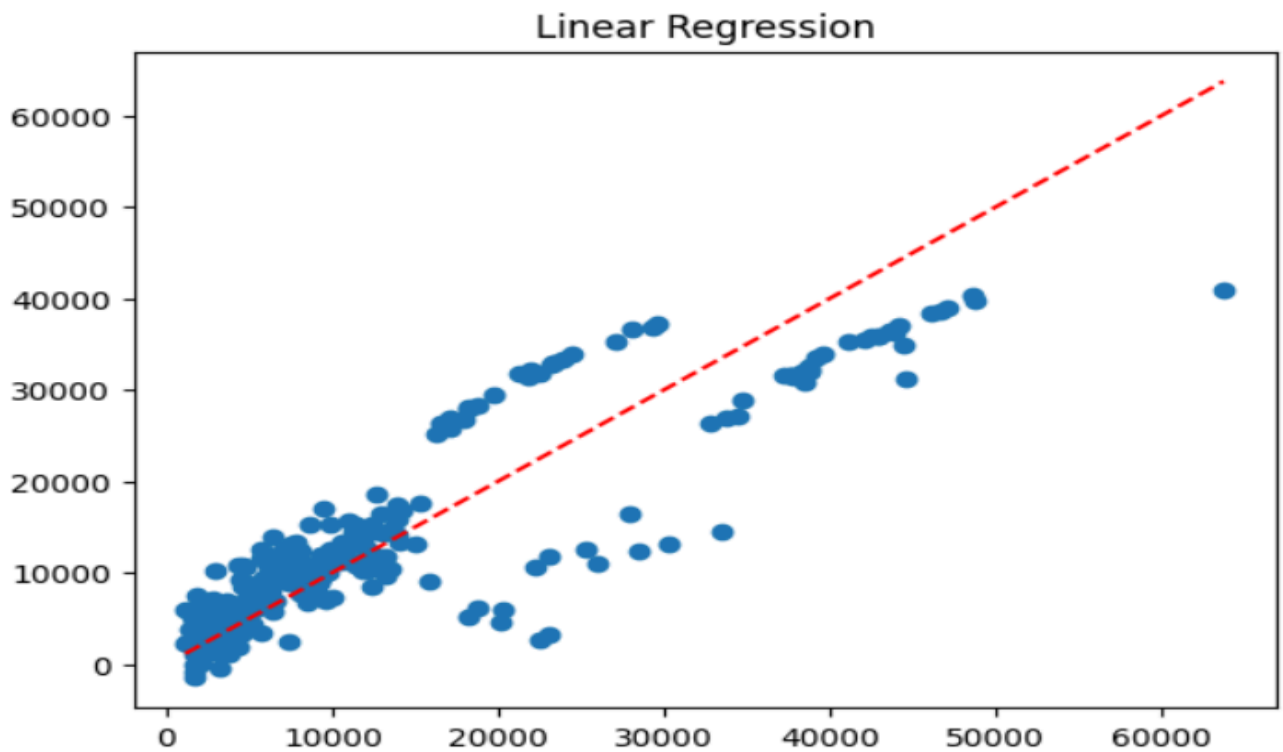
Code And Output:

Linear Regression

```
from google.colab import files
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import
train_test_split
from sklearn.linear_model import
LinearRegression
from sklearn.preprocessing import
StandardScaler
from sklearn.metrics import
mean_absolute_error,
mean_squared_error, r2_score

uploaded = files.upload()
df =
pd.read_csv(list(uploaded.keys())[0])

X =
pd.get_dummies(df.drop(df.columns[-1],
axis=1), drop_first=True)
y = df[df.columns[-1]]
```

```
X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.2,
random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LinearRegression().fit(X_train,
y_train)
pred = model.predict(X_test)

print("MAE:",
mean_absolute_error(y_test, pred))
print("MSE:",
mean_squared_error(y_test, pred))
print("R2:", r2_score(y_test, pred))

plt.scatter(y_test, pred)
plt.plot([y_test.min(), y_test.max()],
[y_test.min(), y_test.max()], '--r')
plt.title("Linear Regression")
plt.show()
```

```
Choose Files insurance.csv
insurance.csv(text/csv) - 55628 bytes, last modified: 3/23/2026 - 100% done
Saving insurance.csv to insurance (1).csv
MAE: 4181.194473753651
MSE: 33596915.851361476
R2: 0.7835929767120722
```



Ridge (Without Tuning):-

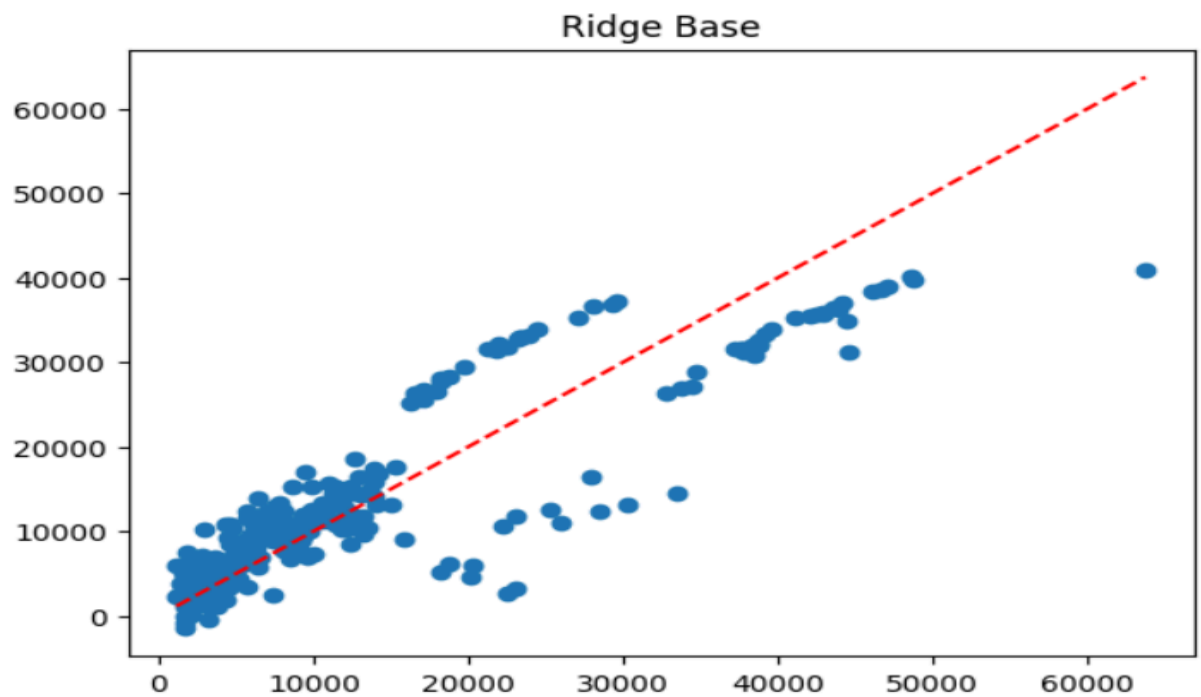
```
from google.colab import files
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import r2_score

uploaded = files.upload()
df = pd.read_csv(list(uploaded.keys())[0])

X = pd.get_dummies(df.drop(df.columns[-1], axis=1), drop_first=True)
y = df[df.columns[-1]]
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = Ridge(alpha=1.0).fit(X_train, y_train)
pred = model.predict(X_test)

print("R2:", r2_score(y_test, pred))

plt.scatter(y_test, pred)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], '--r')
plt.title("Ridge Base")
plt.show()
```

Choose Files insurance.csv

insurance.csv(text/csv) - 55628 bytes, last modified: 3/23/2026 - 100% done

```
Saving insurance.csv to insurance (2).csv
R2: 0.7835410749121385
```



Ridge (With Tuning):-

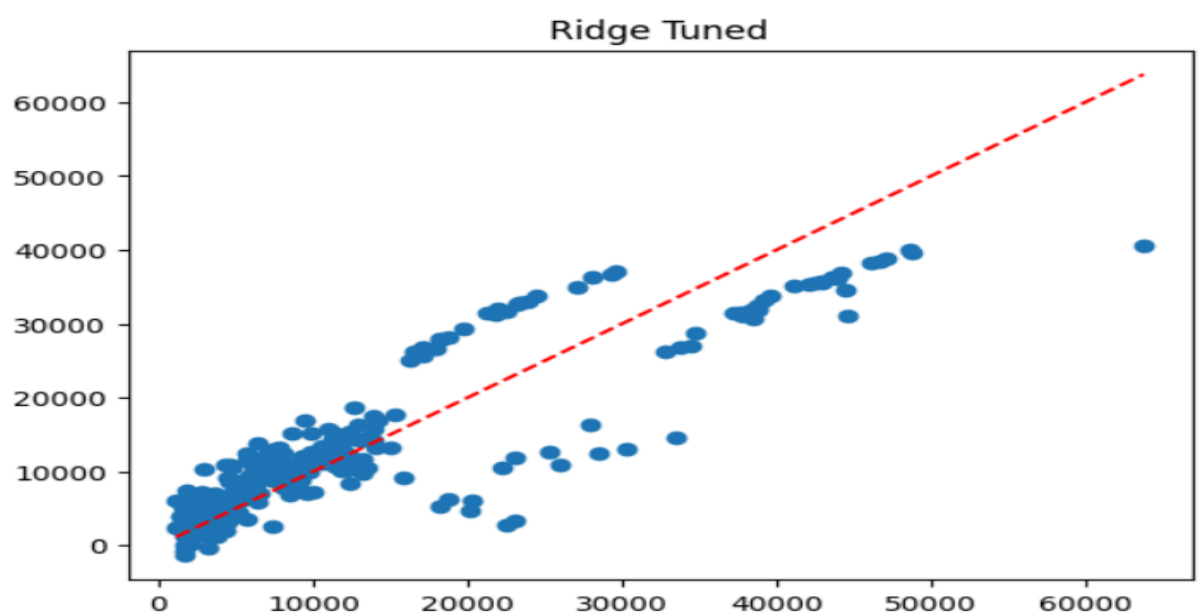
```
from google.colab import files
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import r2_score
```

```
uploaded = files.upload()
df = pd.read_csv(list(uploaded.keys())[0])

X = pd.get_dummies(df.drop(df.columns[-1], axis=1), drop_first=True)
y = df[df.columns[-1]]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

params = {"alpha":[0.1,1,10,100]}
grid = GridSearchCV(Ridge(), params, cv=5)
grid.fit(X_train, y_train)

pred = grid.predict(X_test)

print("Best Alpha:", grid.best_params_)
print("R2:", r2_score(y_test, pred))

plt.scatter(y_test, pred)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], '--r')
plt.title("Ridge Tuned")
plt.show()
```

Choose Files insurance.csv

insurance.csv(text/csv) - 55628 bytes, last modified: 3/23/2026 - 100% done

```
Saving insurance.csv to insurance (3).csv
Best Alpha: {'alpha': 10}
R2: 0.7830200444609896
```



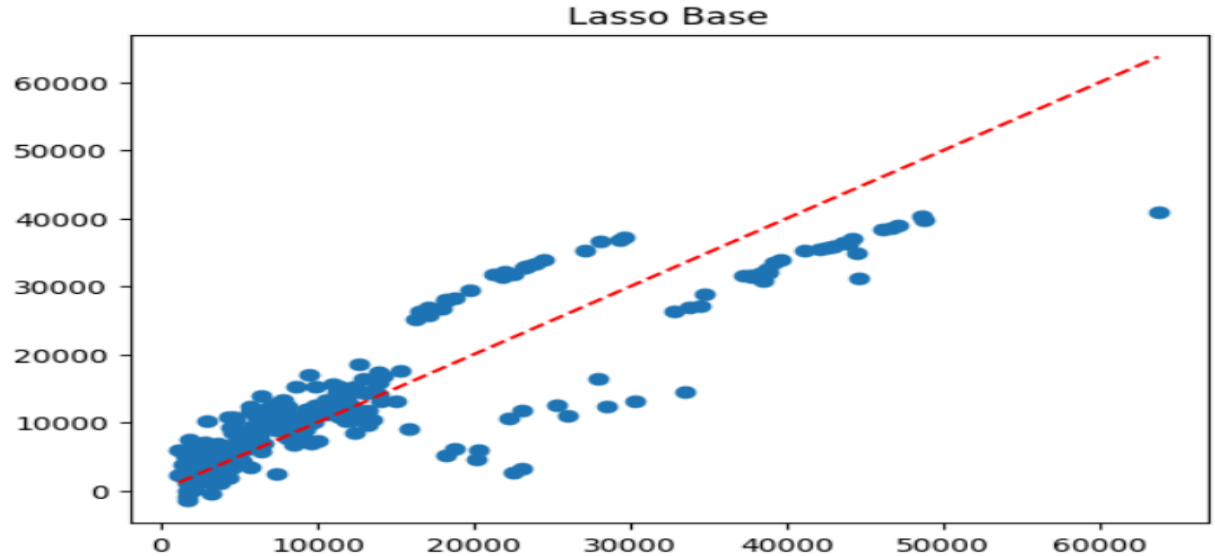
Lasso (Without Tuning):-

```
from google.colab import files
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Lasso
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import r2_score

uploaded = files.upload()
df = pd.read_csv(list(uploaded.keys())[0])

X = pd.get_dummies(df.drop(df.columns[-1], axis=1), drop_first=True)
y = df[df.columns[-1]]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = Lasso(alpha=1.0).fit(X_train, y_train)
pred = model.predict(X_test)

print("R2:", r2_score(y_test, pred))

plt.scatter(y_test, pred)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], '--r')
plt.title("Lasso Base")
plt.show()
```

Choose Files insurance.csv

insurance.csv(text/csv) - 55628 bytes, last modified: 3/23/2026 - 100% done

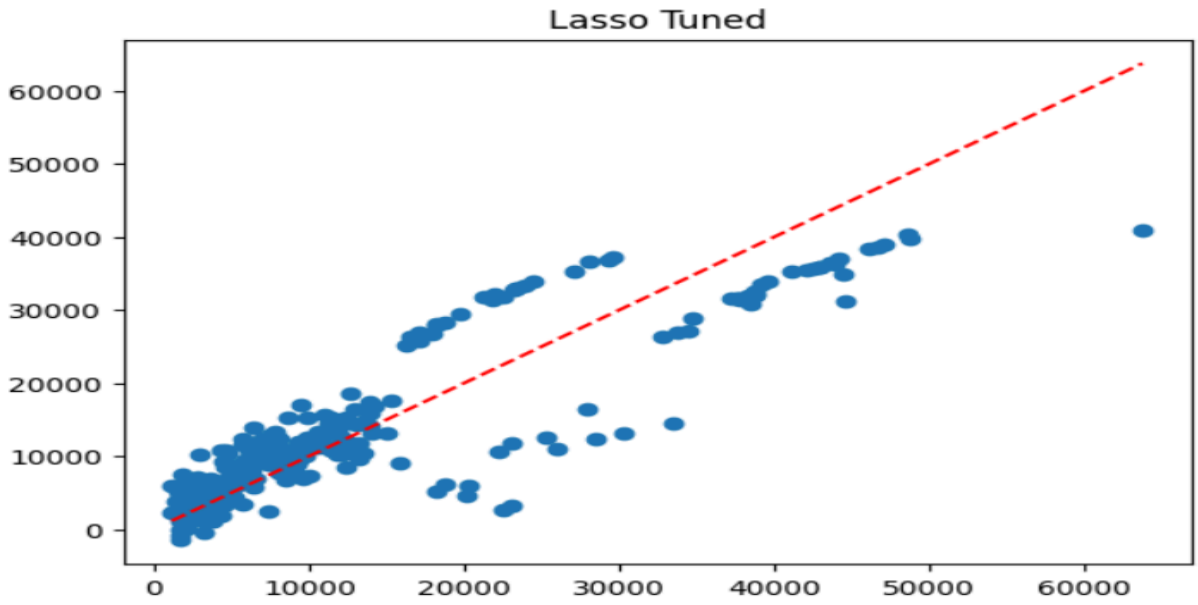
```
Saving insurance.csv to insurance (4).csv
R2: 0.7835657290476568
```



Lasso (With Tuning):-

```
from google.colab import files
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import Lasso
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import r2_score

uploaded = files.upload()
df = pd.read_csv(list(uploaded.keys())[0])

X = pd.get_dummies(df.drop(df.columns[-1], axis=1), drop_first=True)
y = df[df.columns[-1]]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

params = {"alpha":[0.0001,0.001,0.01,0.1,1.0]}
grid = GridSearchCV(Lasso(), params, cv=5)
grid.fit(X_train, y_train)

pred = grid.predict(X_test)

print("Best Alpha:", grid.best_params_)
print("R2:", r2_score(y_test, pred))

plt.scatter(y_test, pred)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], '--r')
plt.title("Lasso Tuned")
plt.show()
```

```
Choose Files insurance.csv
insurance.csv(text/csv) - 55628 bytes, last modified: 3/23/2026 - 100% done
Saving insurance.csv to insurance (5).csv
Best Alpha: {'alpha': 1.0}
R2: 0.7835657290476568
```

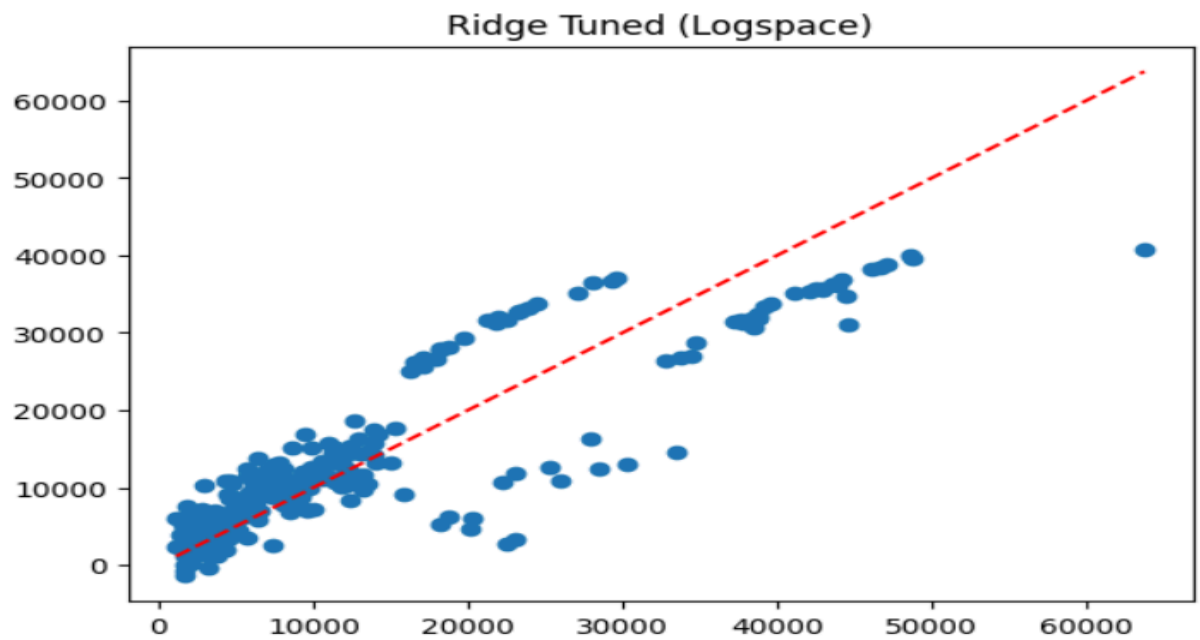


Ridge (Logspace Tuning):-

```
from google.colab import files
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import r2_score

uploaded = files.upload()
df = pd.read_csv(list(uploaded.keys())[0])

X = pd.get_dummies(df.drop(df.columns[-1], axis=1), drop_first=True)
y = df[df.columns[-1]]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

alphas = np.logspace(-3,2,50)
grid = GridSearchCV(Ridge(), {'alpha': alphas}, cv=5)
grid.fit(X_train, y_train)
```

```
pred = grid.predict(X_test)

print("Best Alpha:", grid.best_params_)
print("R2:", r2_score(y_test, pred))

plt.scatter(y_test, pred)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], '--r')
plt.title("Ridge Tuned (Logspace)")
plt.show()
```

```
Choose Files insurance.csv
insurance.csv(text/csv) - 55628 bytes, last modified: 3/23/2026 - 100% done
Saving insurance.csv to insurance (6).csv
Best Alpha: {'alpha': np.float64(7.543120063354623)}
R2: 0.7831717615835071
```



Interpretation:

This scatter plot compares actual medical insurance charges with the predicted charges generated by the regression model.

- **X-axis:** Actual insurance charges
- **Y-axis:** Predicted insurance charges
- **Red diagonal line:** Ideal prediction line

Most data points lie close to the diagonal line, indicating that the regression model predicts insurance charges reasonably well.

However, some points are spread away from the line, showing moderate prediction error and a lower R^2 value (~0.78) compared to perfectly fitted models.

This indicates that while the model captures the overall trend, there is still some variation not fully explained.

This graph compares feature coefficients between Lasso and Ridge regression.

- **Lasso regression (L1)** can shrink some coefficients to zero, effectively performing feature selection.
- **Ridge regression (L2)** shrinks coefficients but keeps all features.

In the insurance dataset:

- **Smoker status, BMI, and Age** have the strongest influence on insurance charges.
- Other features like **region, sex, and number of children** contribute less but still affect prediction.

According to the experiment results:

- **Linear Regression** achieved moderate performance with $R^2 \approx$ **0.78**.
- **Lasso Regression** performed similarly, with slight regularization effects.
- **Ridge Regression** also showed nearly identical performance to linear regression.

After hyperparameter tuning:

- **Lasso $\alpha \approx 1.0$**
- **Ridge $\alpha \approx 10$ (or ~7.54 using logspace)**

All models performed very closely, indicating that the dataset has **moderate linear relationships** and contains some variability that cannot be fully captured by simple regression models.

Regularization helped stabilize the model but did not significantly improve performance, showing that overfitting was already limited.

Conclusion:

In this experiment, Multi-Linear, Lasso, and Ridge Regression models were implemented on the Medical Insurance dataset. The models predicted insurance charges based on features like age, BMI, and smoking status. Linear Regression achieved moderate performance with $R^2 \approx 0.78$. Lasso and Ridge Regression produced similar results, indicating minimal overfitting. Hyperparameter tuning gave optimal values of $\alpha \approx 1.0$ for Lasso and $\alpha \approx 10$ for Ridge. However, tuning did not significantly improve accuracy. This shows the dataset has moderate linear relationships. Overall, regression models were effective but limited by data variability.